Kandidat nummer: 79, 51

Høyskolen Kristiania

PGR208

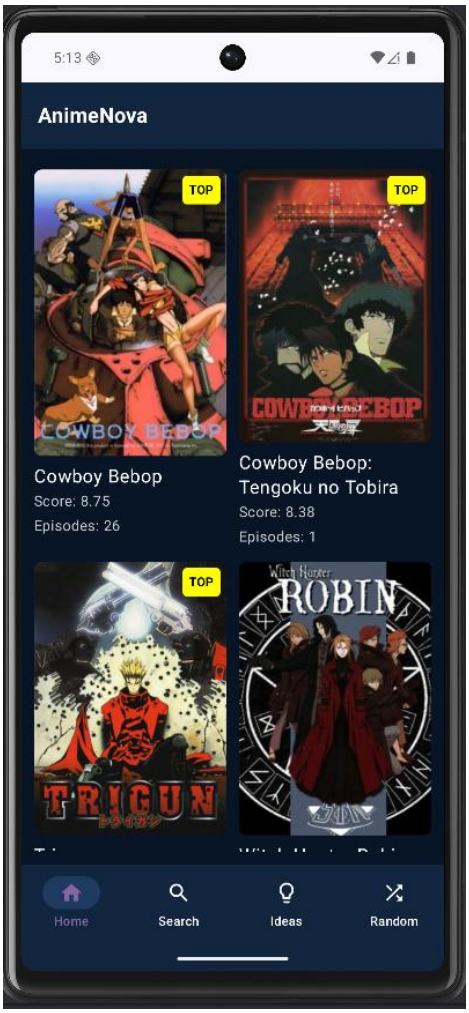
Eksamen rapport

App-dokumentasjon (25%)

H25

Del 1. Redegjørelse av app per skjerm

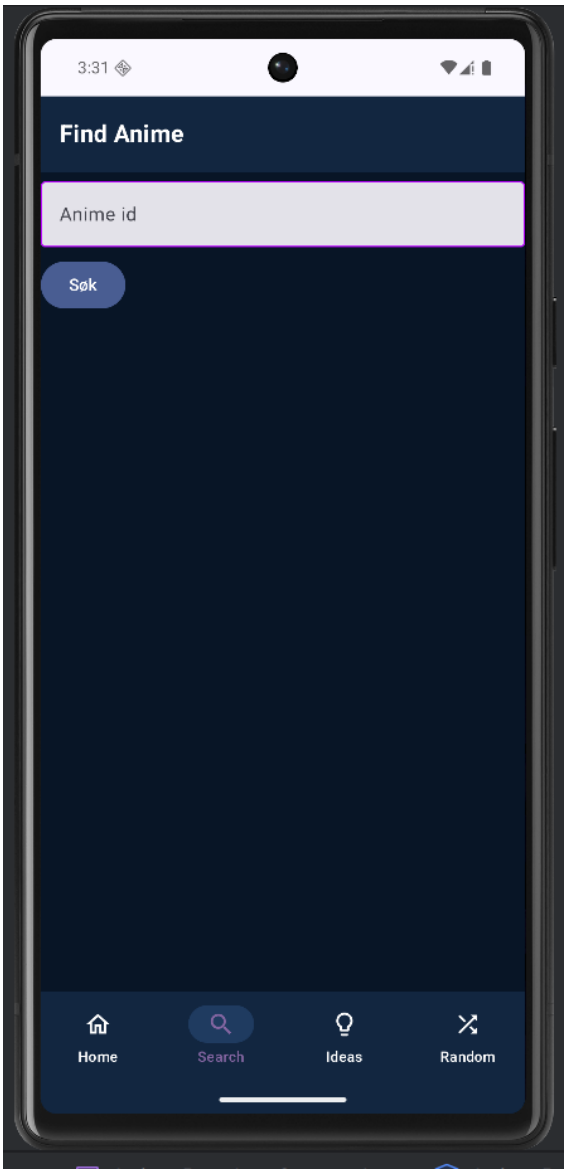
Skjerm: 1



Skjerm 1 viser en liste over anime hentnet fra APIet. Målet med denne skjermen er å gi brukeren en oversiktlig og ryddig presentasjon av flere anime samtidig, samtidig som den følger den visuelle strukturen vi har etablert for resten av appen. For å hente data fra API-et bruker vi MVVM arkitekturen med et repository og en dedikert ViewModel. ViewModelen henter anime-listen ved oppstart og oppdaterer UI-et gjennom observbare state-verdier. Dette gjør at innholdet vises automatisk så snart dataene er lastet inn. I tillegg håndteres ulike tilstander, som «Laster..», feilmeldinger eller tomt resultat, slik at brukeren hele tiden får

tydelig tilbakemelding. Layouten er bygget med en felles AppBackground og Appheader øverst med tittelen «AnimeNova». Under headeren vises anime-listen gjennom en LazyColumn der hvert element vises gjennom komponenten AnimeItem. Hvert anime kort presenterer bilde, tittel, score og antall episoder for serien. For bildehåndtering bruker vi Coil sin Asyncimage, som gjør det enkelt å laste inn bilder direkte fra URL-er uten ekstra konfigurasjon. Som en liten tilleggsfunksjon har vi lagt inn en «TOP» badge på anime med høy score (over 8.0). Denne vises som et gult merke øverst i hjørnet av bildet og gir brukeren en rask indikasjon på hvilke anime som er populære, og mer informasjon på en visuell måte uten at skjermen blir mer komplisert å bruke.

Skjerm 2:



Før brukeren gjør en handling på skjermen.

Etter at brukeren har lagt inn et id nummer

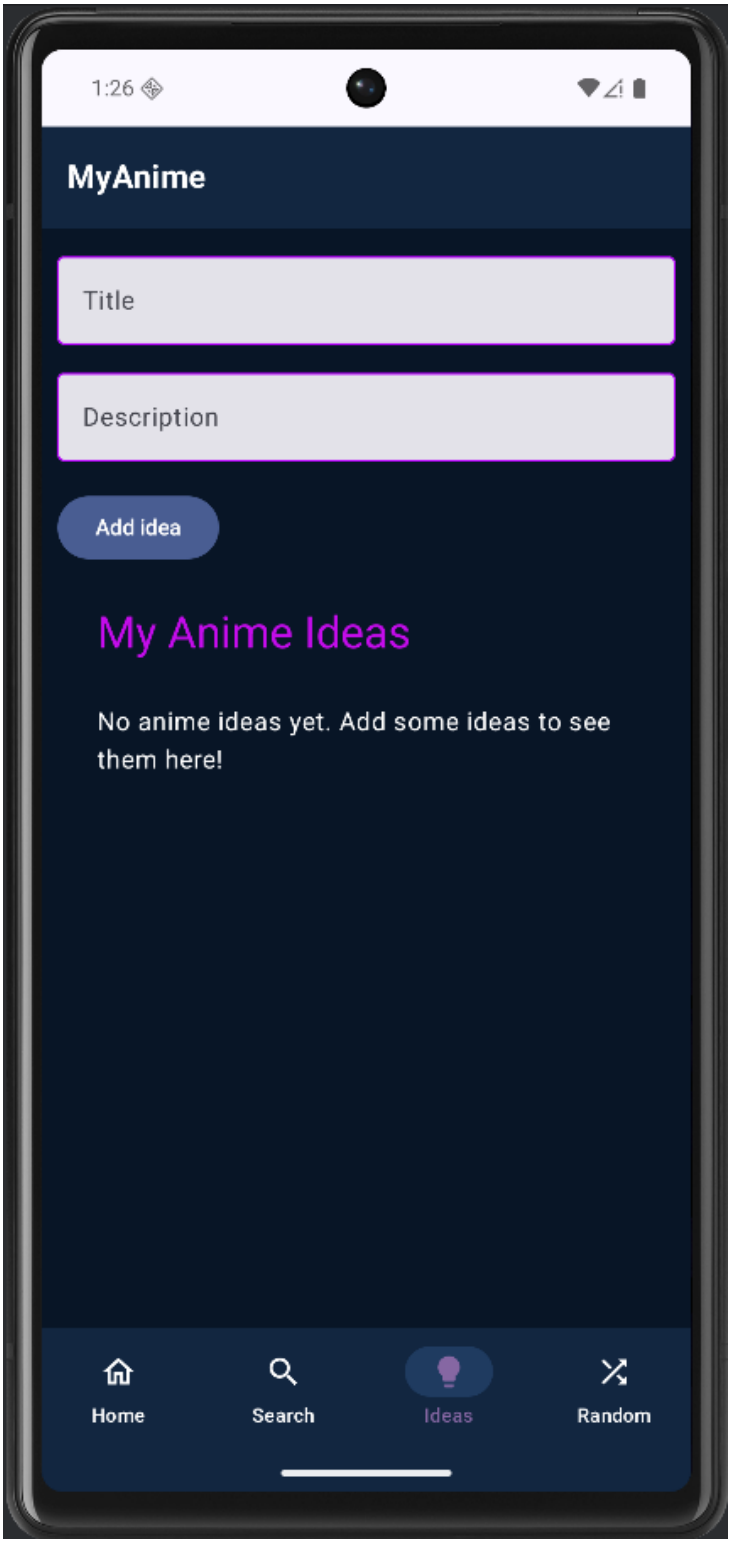
Etter at brukeren har lagt inn et id som ikke finnes i anime API-et.

Skjerm 2 lar brukeren søke etter en anime basert på en spesifikk id fra Jikan-APIet. Før brukere gjør noe, vises et enkelt inputfelt for å skrive inn en anime-id sammen med en

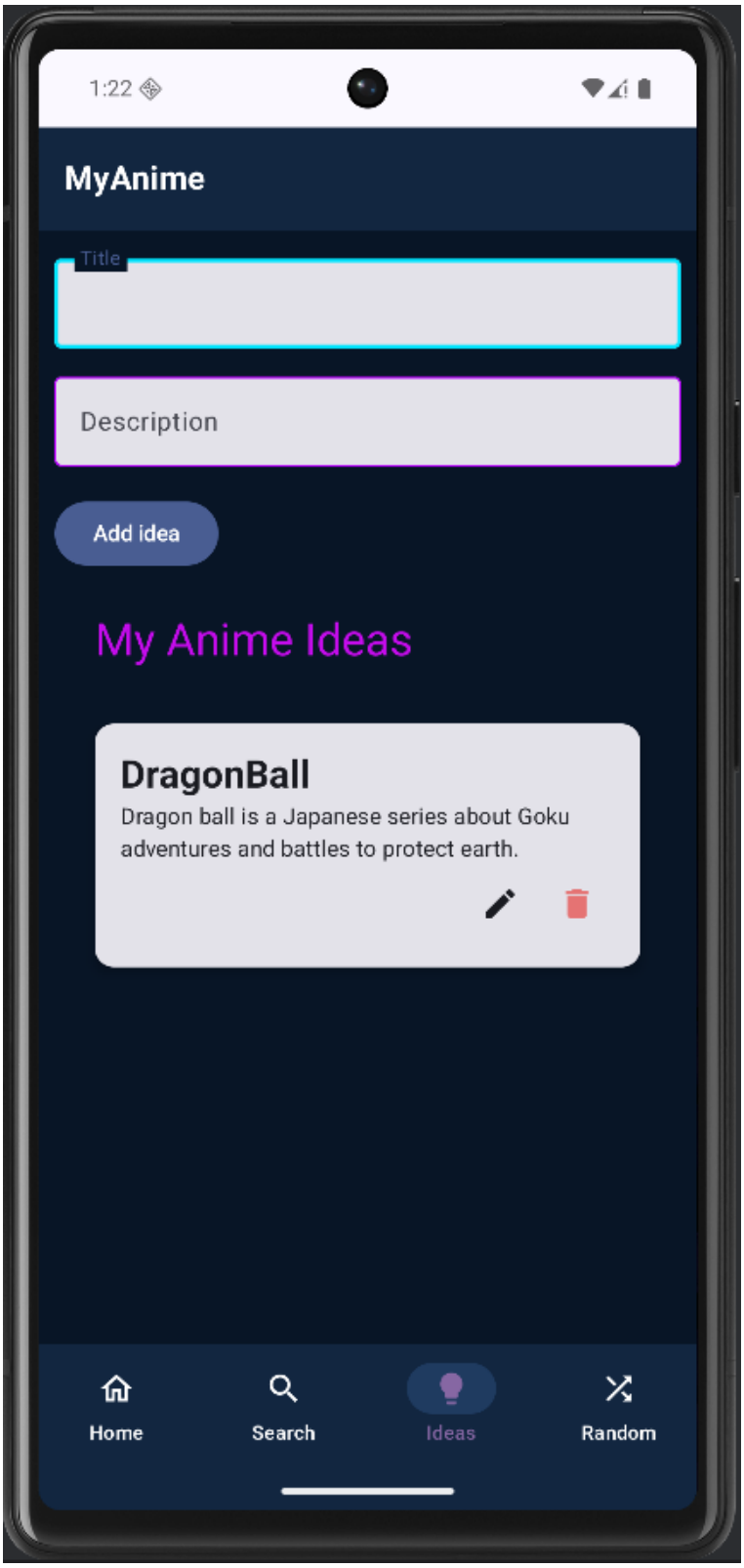
søkeknapp. Skjermen har tre tydelige tilstander: før søk, resultat funnet eller feilmelding. Dette gir en ren og enkel brukeropplevelse. Når brukeren skriver inn en id og trykker «søk» så kalles funksjonen search() i AnimeSearchViewModel, som håndterer hele søkelogikken for denne skjermen. Det første som skjer, er at id-en valideres. Dersom brukeren skriver inn noe som ikke kan konverteres til et tall så avbrytes søket og det vises en feilmelding. Dette hindrer unødvendige API-kall og gir brukeren rask tilbakemelding. Hvis id-en er gyldig så starter ViewModel et coroutine-kall som henter anime basert på id via getAnimeById() i AnimeRepository. Repositoryet bruker AnimeService til å sende forespørselen til Jikan-APIet og returnerer enten et animeobjekt eller null. Når resultatet kommer tilbake så oppdateres state-verdiene i ViewModel som igjen oppdaterer UI-et automatisk gjennom mutableStateOf.

Dersom en anime med valgt if finnes, vises resultatet på skjermen gjennom AnimeItem, den samme UI-komponenten som brukes i skjerm 2. Dette gir både gjenbruk av kode og en konsistent visuell stil i appen. Hvis id-en ikke finnes i API-et så vises en tydelig feilmelding slik at brukeren forstår hva som gikk galt. Underveis vises også en «laster» status mens API-kallet pågår.

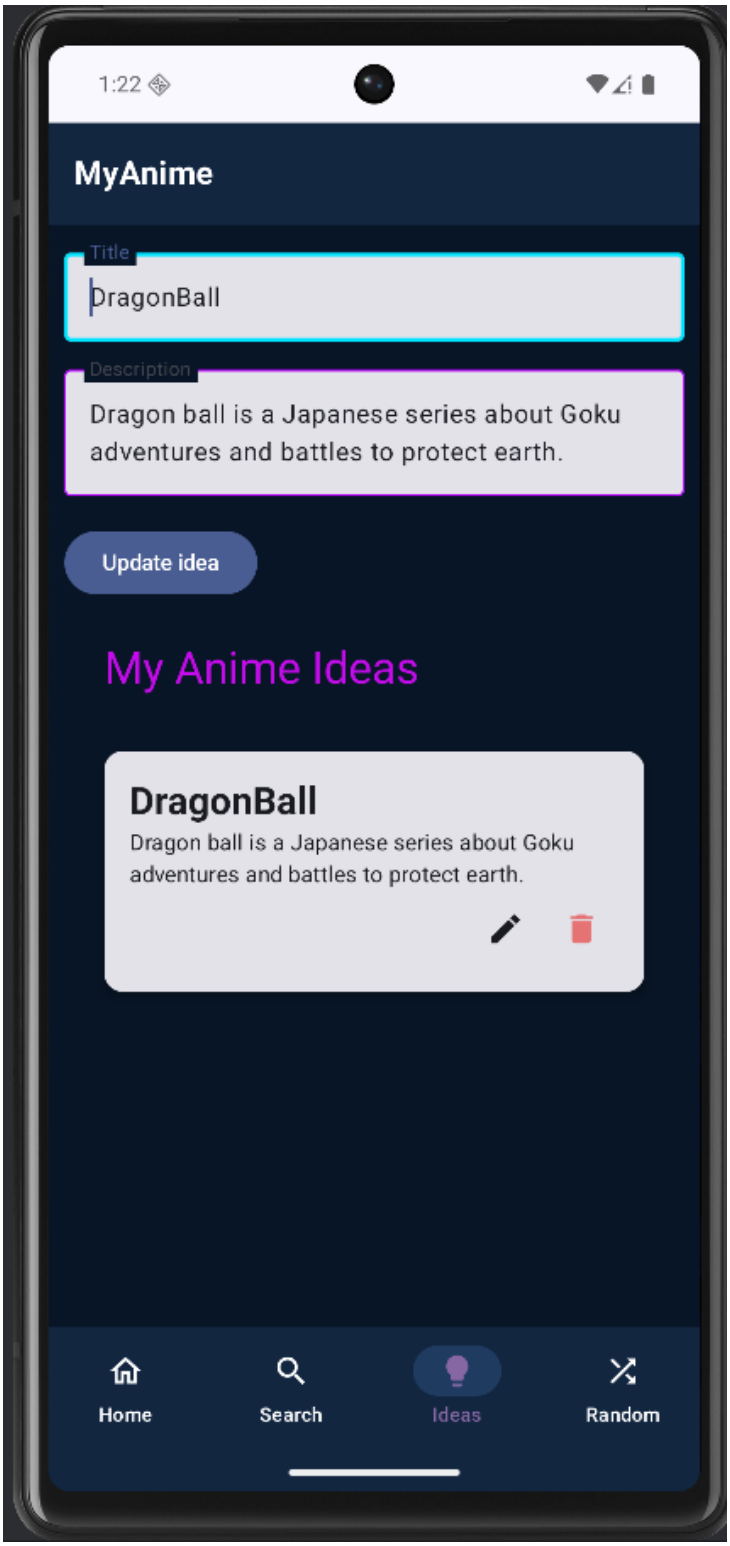
Skjerm 3:



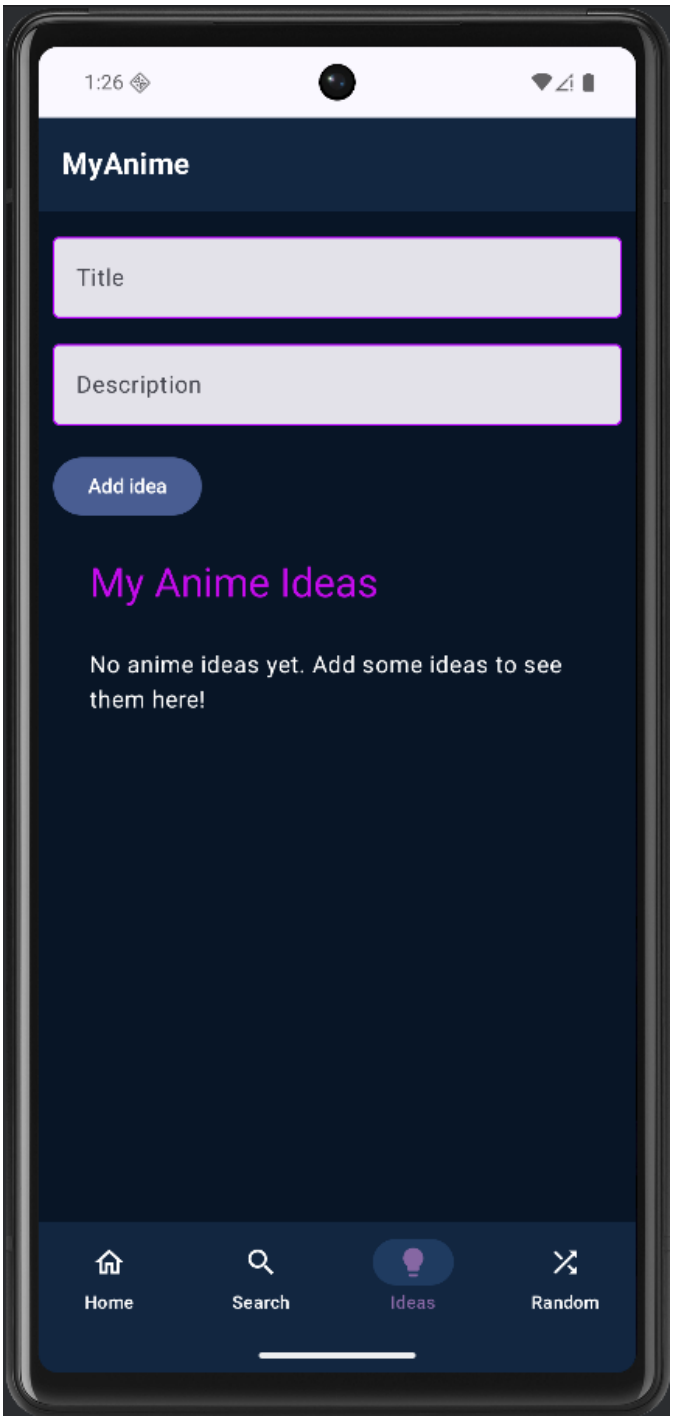
Før brukeren gjør en handling på skjermen.



Etter at brukeren har lagt til navn og beskrivelse på ideen.



Etter at brukeren har trykket på Rediger knappen. (Inputfeltene blir som redigeringsmodus, og knappen endrer seg)



Etter at brukeren har slettet Anime ide

Skjerm 3 viser oversikt over brukerens Anime Ideer. Før brukeren gjør noe, så vises en enten en liste med ideer som er hentet fra databasen, eller en tom tilstand hvor det står «No Anime ideas yet.» hvis ingen ideer er lagret. Brukeren har mulighet til å legge til tittel og beskrivelse for en ide i inputfeltene, og dette blir lagt til ved at brukeren trykker på «Add idea» knappen som da kaller funksjonen addAnimeidea I AnimeIdeasViewmodel. Disse ideene blir da lagt til

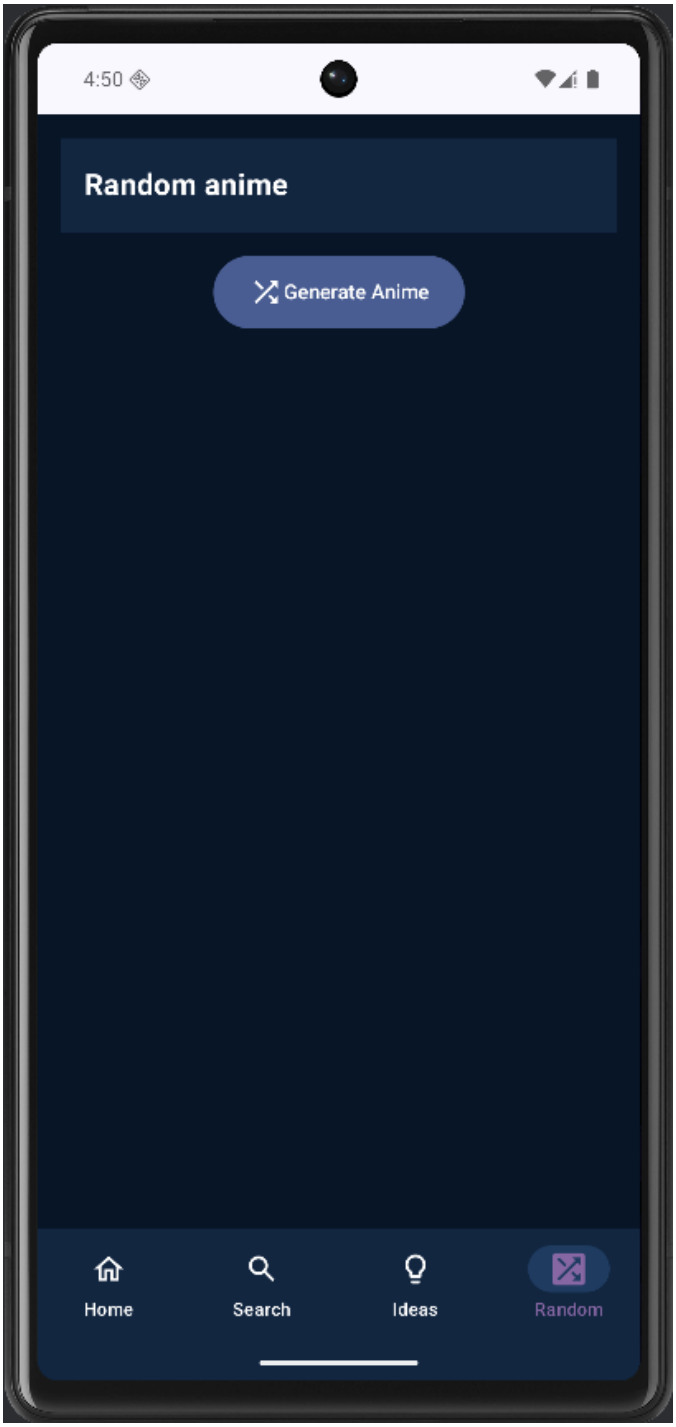
i databasen via AnimeDbRepository. Vi har også valgt å bruke room unique constraint på title feltet i anime idea entiteten, slik at vi kan sørge at brukeren ikke kan lagre to ideer med samme tittel. Listen blir da oppdatert via loadAnimeIdeas i AnimeIdeaViewmodel. Grunnen til dette er fordi det gir en god brukeropplevelse når brukeren kan se resultatet av handlingen.

Brukeren kan også redigere en ide, ved å trykke på redigerings ikonet. Når brukeren trykker på ikonet sjekkes det at editingID ikke er null, og inputfeltene tittel og beskrivelse settes til verdiene fra den valgte ideen, og editingID settes til ideens ID. Grunnen til dette er fordi at brukeren skal kunne se den eksiterende tittelen og beskrivelsen i inputfeltene og redigere dem der direkte. Når brukeren trykker på knappen «Update idea» blir det opprettet et nytt AnimeIdea objekt, med de endrede verdiene og samme id, og deretter kaller på updateAnimeIdea i AnimeIdeasViewmodel. Endringene blir sendt videre til database repository som utfører selve oppdateringen. Deretter blir loadAnimeIdeas funksjonen kalt for å hente listen på nytt, slik at brukeren kan se det.

Når brukeren trykker på slett ikonet kalles deleteAnimeIdea i viewModel, som da setter isDeleted til true på ideen i databasen via AnimeDbRepository. Det ser ut som at det er slettet for brukeren, men id blir fortsatt bevart i databasen. Selv om både hard delete og soft delete av data begge har fordeler valgte vi å gå for en soft delete for denne eksamenen, grunnen til dette er fordi 1. Vi ønsket å ha historikken av alle id og data, 2. Fysisk sletting kan skape hull i Id sekvensen, 3. Det er mulighet for å enkelt gjenopprette poster hvis brukeren sletter ved et uhell.

.

Skjerm 4:



Før brukeren gjør en handling på skjermen.

Etter at brukeren trykker på generate Anime (Den viser en tilfeldig Anime).

Skjerm 4 viser en tilfeldig valgt anime henter fra API. Vi har valgt denne tilleggsfunksjonen fordi logikken bak å hente ett tilfeldig element ved hjelp av .random, er enkel og effektiv å kode. Det er også for å gi brukeren en gøy og uforutsigbar opplevelse, slik at hver gang brukeren trykker på knappen, kan de få et nytt forslag uten å måtte bla gjennom hele listen. Denne funksjonen er også noe vi har sett på flere nettsider som tilbyr anime Serier.

Når brukeren trykker på knappen «Give me random anime» kalles funksjonen generateAnime() i AnimeRandomViewModel. Dette gjør at funksjonen kjører asynkront via viewModelScope.Launch, dette er for at skjermen ikke skal fryses mens dataen lastes. Deretter hentes hele listen av anime via AnimeRepository.getAnimeList(). Dersom listen ikke er tom, velges kun en tilfeldig anime fra listen ved hjelp av .random(). Denne blir satt til randomAnime.value, .value er den faktiske verdien i mutableStateOf, og når denne endres blir UI oppdatert automatisk for å vise den nye anime.

Dersom listen er tom settes randomAnime.value til null. Når bruker trykker på knappen flere ganger, starter viewModel et nytt API-kall før forrige API-Kall var ferdig og errorMessage.Value blir satt til en feilmelding hvor det står «Fant ingen anime i lista, prøv igjen», dette er noe som vises til brukeren, slik at brukeren skjønner at de kan trykke igjen å prøve, istedenfor å vise en tom skjerm..

Del 2. Redegjørelse av planlegging og utvikling.

Hvordan har dere planlagt løsningen?

Vi planlagte løsningen ved å først visualisere hvordan appen vår skulle se ut, før vi tenkte over logikken. Vi lagde fire enkle skisser for hver skjerm, som hjalp oss med å visualisere hvordan løsningen vår ville se ut. Dette gjorde det lettere for oss å planlegge hva slags logikk vi skal ha.

Hvordan har dere lagt opp arbeidet og fordelingen av arbeidet?

Etter at vi hadde tegnet skissene og diskutert ideene, startet vi med å opprette pakkene og fil strukturen for prosjektet vårt. Dette gjordet det også enklere for oss til å organisere arbeidet. Vi valgte å jobbe i separate prosjekter først, og deretter flette arbeidet underveis, samtidig som vi har forklart koden slik at begge fikk en god forståelse av hele prosjektet.

Hvordan kommet frem til avgjørelser?

Vi har tatt avgjørelser på en gjensidig måte, der vi begge har prioritert løsninger som er enkle å forstå og implementere. Vi har diskutert og utforsket for å finne de løsningene vi mente passet best til prosjektet vårt.

Hvordan gikk samarbeidet?

Samarbeidet gikk veldig fint gitt at vi alltid var i dialog, men også fysisk møttes for å kode sammen, diskutere og utveksle ideer. Vi hjalp hverandre, selv om vi kodet i separat prosjekt i egne ansvarsfiler. Dersom det dukket opp en error så vi på det sammen og lærte hvordan man skulle løse disse sammen. Dette gjorde samarbeidet ganske flytende. I tillegg fordelte vi alltid oppgaver til hverandre og hadde som mål om å løse det i løpet av møtet vårt, noe som førte til at samarbeidet vårt var effektivt.

Spesifikasjoner av kilder som ble brukt i kode utviklingen

Room-database

Vi har tilpasset koden for vårt prosjekt ved å legge til funksjonalitet som håndtering av soft delete som er inspirert av en anbefalt løsning på StackOverFlow: https://stackoverflow.com/questions/58088551/how-to-undo-a-deletion-made-using-room-persistent-library

Her blir det anbefalt å bruke flag column altså våre «isDelete = 1» for å markere rader som slettet uten å fjerne de fysisk.

Vi har også benyttet OnConflictStrategy.Replace strategien i Dao for å håndtere duplikat i titler, her har vi blitt direkte inspirert av koden i denne kilden: https://stackoverflow.com/questions/56231855/room-database-onconflict-onconflictstrategy-replace-not-working

I AnimeIdea filen valgte vi å bruke unik constraints på tittel, slik at ingen rader kan ha samme tittel. Inspo fra: https://code.luasoftware.com/tutorials/android/android-room-add-unique-contraint-to-table

ViewModel for AnimeIdeas

Funksjonen «loadAnimeIdeas» fra ViewModel for AnimeIdeas er inspirert fra: https://proandroiddev.com/the-best-way-to-load-data-in-viewmodels-a112ced54e07 og https://stackoverflow.com/questions/77508740/how-to-update-stateflow-inside-the-viewmodel .

API

Jikan-APIet hadde en mer detaljert JSON-struktur enn eksempelet i forelesningen, spesielt for bilder. Bilde-URLen ligger ikke direkte i objektet, men organisert i flere nivåer (images – jpg/webp – image_url). For å modellere dette korrekt brukte vi den offisielle dokumentasjonen til Jikan og testet responsen direkte fra endepunktene for å se hele strukturen. Inspo: https://docs.api.jikan.moe/

Vi brukte også støttekilder fra Stack Overflow om hvordan man håndterer flernivå JSON i Retrofit, blant annet fra denne kilden:

https://stackoverflow.com/questions/32942661/how-can-retrofit-2-0-parse-nested-json-object?

I tillegg opplevde vi problemer med at raske gjentatte API-kall ga tomme resultater, spesielt i skjerm 4 hvor det blir generert en tilfeldig anime. For å løse dette flyttet vi ansvaret for API-kallet inn i ViewModel og hentet hele anime-listen før vi får et tilfeldig element. Løsningen samsvarer også med anbefalte praksiser for bruk av coroutines og StateFlow, slik som i denne kilden fra ProAndroidDev. https://proandroiddev.com/the-best-way-to-load-data-in-viewmodels-a112ced54e07

NB!
Vi er klar over at det ikke er anbefalt å bruke stor forbokstav på pakker, men når vi prøvde å endre fra Room til room, så bare ville den ikke endre på seg.

Kilde liste

- Android Developers (n.d) *Configure text fields* https://developer.android.com/develop/ui/compose/text/user-input?textfield=state-based
- Android Developers (n.d) *Migrate to state-based text fields.* https://developer.android.com/develop/ui/compose/text/migrate-state-based
-
- Android Developers (n.d) *State and Jetpack Compose.* *https://developer.android.com/develop/ui/compose/state*
- Android Developers (n.d) *Define data using Room entities*https://developer.android.com/training/data-storage/room/defining-data
- CodingZest(2024) *Android Jetpack Compose Tutorial#5 How to use TextField in JetPack Compose* https://www.youtube.com/watch?v=iRxPUrwnaTo&t=495s¨
- Jikan. (n.d)*JSON error response.*Jikan APIhttps://docs.api.jikan.moe/#/section/information/rate-limiting

- Kella, S. (2025) How to properly use stateflow in viewmodel to power your compose UI. *Medium* https://medium.com/kotlin-android-chronicle/how-to-properly-use-stateflow-in-viewmodel-to-power-your-compose-ui-1a34791a3718

- Klitsie, J. (2024) The best way to loas data in ViewModels. *Medium* https://proandroiddev.com/the-best-way-to-load-data-in-viewmodels-a112ced54e07

- Stackoverflow(2024*) add unique constraint in room database to multiple column* https://stackoverflow.com/questions/48962106/add-unique-constraint-in-room-database-to-multiple-column

- Stackoverflow(2022*)lateinit property has not been initialized error occur*https://stackoverflow.com/questions/71688525/lateinit-property-has-not-been-initialized-error-occur

- Stackoverflow(2023)*How to update StateFlow inside the ViewModel* https://stackoverflow.com/questions/77508740/how-to-update-stateflow-inside-the-viewmodel

- Stackoverflow(2022) *Update Row if it Exists Else Inser Logic with Entity FrameWork* https://stackoverflow.com/questions/5557829/update-row-if-it-exists-else-insert-logic-with-entity-framework

- Stackoverflow(2023*) room update(or insert if exist) rows and return count changed rows*https://stackoverflow.com/questions/48519896/room-update-or-insert-if-not-exist-rows-and-return-count-changed-rows

- StackOverflow(2022) *How to undo deletion made using Room persistent library?* https://stackoverflow.com/questions/58088551/how-to-undo-a-deletion-made-using-room-persistent-library